
```
>>> print(employeeRecord)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
❶ NameError: name 'employeeRecord' is not defined

>>> 42 - 'hello'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
❷ TypeError: unsupported operand type(s) for -: 'int' and 'str'
```

This example has a typo in the variable `employeeRecord`, causing an error ❶. Because the identifier `employeeRecord` in `NameError: name 'employeeRecord' is not defined` is specific to your code, you might want to instead search for **python “NameError: name” “is not defined”**. In the last line, the `'int'` and `'str'` part of the error message ❷ seems to refer to the `42` and `'hello'` values, so truncating the search to **python “TypeError: unsupported operand type(s) for”** would avoid including parts particular to your code. If these searches don’t yield useful results, try including the full error message.

Preventing Errors with Linters

The best way to fix mistakes is to not make them in the first place. Lint software, or *linters*, are applications that analyze your source code to warn you of any potential errors. The name references the small fibers and debris collected by a clothes dryer’s lint trap. Although a linter won’t catch all errors, *static analysis* (examining source code without running it) can identify common errors caused by typos. (Chapter 11 explores how to use type hints for static analysis.) Many text editors and integrated development environments (IDEs) incorporate a linter that runs in the background and can point out problems in real time, such as in [Figure 1-2](#).